

LLD Notes — Phase 3, Part 1

Strategy · Factory · Singleton · Java concurrency foundations | Concepts, your doubts, your mistakes, interview answers

Colour key. **Blue = a doubt you asked** (with the explanation that made it click) · **Red = a mistake you made** (wrong → right) · **Green = rule to remember** · **Amber = how to say it in an interview**. Before a revision session, read every red box and the mistake summary first.

Contents

0. What a pattern is, and how interviewers use them
1. Strategy — structure · strategies needing different data · when not to use · Strategy vs State · lambdas vs classes · exercise 16 (cache eviction)
2. Factory — simple · registry + Supplier · shared vs fresh instances · Factory Method · Abstract Factory · exercise 17 (exporters)
3. Concurrency foundations — caches & visibility · reordering · atomicity · volatile · synchronized · Atomic classes · class initialisation · locks · ConcurrentHashMap
4. Singleton — all six versions in full · which to use · breaking it · reflection · why it's criticised · Spring beans · exercises 18 & 19
5. Mixed exercise 1 (ride fare engine) — decisions, mistakes, reference
6. Your Phase 3 mistake summary · requirement → pattern cheat sheet · interview quick-recall · checklist

0. What a Design Pattern Is

A pattern is a **named, proven solution to a recurring design problem**. Not a library, not code to copy — a shape you recognise and adapt.

- **Vocabulary.** "I'd use a Strategy here" says in four words what would otherwise take two minutes.
- **Tested structure.** The pitfalls have already been found; you don't rediscover them.

How interviewers actually ask. Rarely "explain the Observer pattern". Instead:
"How would you handle different pricing rules?" → **Strategy**
"Where do these objects get created?" → **Factory**
"Every part of the system must see the same counts" → **Singleton** (one shared instance)
The skill is **mapping a requirement to a pattern** — and the equal counter-skill is knowing when a plain class is the right answer.

Frequency reality: Strategy, Factory, Observer, Singleton, Builder, State and Decorator cover roughly 85% of what you'll be asked to produce. Weight practice accordingly.

Phase 3 rhythm (your request): one pattern per session with one exercise → after 3 patterns a mixed exercise → after 4–5 patterns a jumbled exercise with no hints.

1. Strategy

The problem

A behaviour has several interchangeable variants, and you don't want the choice scattered through your code.

```
class ExpenseSplitter {
    public Map<String, Double> split(String type, double amount, List<String> users) {
        if (type.equals("EQUAL")) { ... }
        else if (type.equals("EXACT")) { ... }
        else if (type.equals("PERCENT")) { ... }
    }
}
```

Every new split type reopens this method; it grows without limit; the same branching gets copied wherever splitting happens.

The structure — three roles, always the same

```
// 1. Strategy — the interface
interface SplitStrategy {
    Map<String, Double> split(double amount, List<String> users);
}

// 2. ConcreteStrategy — one per variant
class EqualSplit implements SplitStrategy {
    public Map<String, Double> split(double amount, List<String> users) {
        double share = amount / users.size();
        // every user gets share
    }
}

// 3. Context — holds a strategy, delegates to it
class Expense {
    private SplitStrategy strategy;
    public Expense(SplitStrategy strategy) { this.strategy = strategy; }
    public void setStrategy(SplitStrategy strategy) { this.strategy = strategy; } // swap at runtime

    public Map<String, Double> calculateShares(double amount, List<String> users) {
        return strategy.split(amount, users); // delegate — no idea which one
    }
}
```

When strategies need different input data

Equal split needs nothing extra. Exact split needs amounts. Percentage split needs percentages. Putting all of them in one signature forces every strategy to accept data it doesn't use — an ISP violation.

```
// x every strategy forced to accept everything
Map<String, Double> split(double amount, List<String> users,
    List<Double> exactAmounts, List<Double> percentages);
```

Option A (usually better) — the strategy carries its own data:

```
class PercentageSplit implements SplitStrategy {
    private final List<Double> percentages;
    public PercentageSplit(List<Double> percentages) { this.percentages = percentages; }
    public Map<String, Double> split(double amount, List<String> users) { /* uses this.percentages */ }
}

new Expense(new PercentageSplit(List.of(50.0, 30.0, 20.0)));
```

Option B — pass a request object (`split(SplitRequest r)`). Useful when data varies a lot, but risks becoming a bag of nullable fields. Prefer A.

When to use / when NOT to use

Use it when	Don't use it when
The requirement lists variants ("equal, exact and percentage")	One variant exists and none is planned — indirection with no payoff (YAGNI)
You see if/else or switch on a type	Variants differ only by a number — <code>PercentDiscount(0.10)</code> and <code>(0.20)</code> are ONE class with a parameter
The algorithm may change at runtime	The "variants" always change together with other classes
You need to test variants in isolation	

Say it: "The pricing rules vary independently of the checkout flow, so I'd put them behind a `DiscountPolicy` interface — new rules become new classes and `Checkout` never changes." And when declining: "There's only one split type today, so I'd keep it concrete — `SplitStrategy` is the obvious seam if more appear."

Where it appears

LLD problem	Strategy over
Parking lot	fee calculation, spot allocation
Splitwise	split types
Cache	eviction policy (LRU / LFU / FIFO)
Rate limiter	algorithm (token bucket / sliding window)
Ride sharing	driver matching, surge pricing
Any system	payment methods, notification channels, sorting

In the JDK/Spring: `Comparator` (different orderings, same sort), `RejectedExecutionHandler`, Spring's `PasswordEncoder`.

Strategy vs State (asked often): structurally identical — both hold an interface and delegate. **Strategy:** the *client* picks the algorithm; strategies don't know each other. **State:** behaviour changes as the object's own state changes, and **states trigger the transition to the next state**. If the objects decide what comes next, it's State.

Lambdas for interfaces

Your doubt: "just give me examples of how lambdas are written for interfaces."

```
interface Greeter { String greet(String name); }           // ONE abstract method

class FormalGreeter implements Greeter {                 // class version
    @Override
    public String greet(String name) { return "Good evening, " + name; }
}
Greeter g1 = new FormalGreeter();

Greeter g2 = name -> "Good evening, " + name;             // lambda version – identical meaning
System.out.println(g2.greet("Ram"));                     // Good evening, Ram

// the mapping:
// public String greet(String name) { return "Good evening, " + name; }
//                               name    ->    "Good evening, " + name
// dropped: class, method name, return type, param type, return, braces
```

Shape	Example
Two parameters (parens required)	Calculator add = (a, b) -> a + b;
One parameter (parens optional)	Validator notEmpty = s -> !s.isEmpty();
No parameters	Task hello = () -> System.out.println("hi");
Multiple statements — braces + return	(a, b) -> { double avg = (a + b) / 2; return Math.round(avg); }
Passed straight in as an argument	names.sort((a, b) -> a.compareTo(b)); names.removeIf(s -> s.length() > 4);

```
interface PaymentMethod { void pay(double amount); }
PaymentMethod cash = amount -> System.out.println("Paid " + amount + " in cash");    // works: stateless

// x UPI needs a upiId – a lambda has no fields and no constructor to hold it
// x interface with TWO abstract methods – "not a functional interface", won't compile
// x can't override a default method – a lambda supplies only the one abstract method
```

Lambda or class? Lambda for a stateless single expression (Comparator, simple Predicate). **Class** when the strategy holds configuration or state, overrides lifecycle hooks, needs a documenting name, or needs its own tests. In LLD strategies usually carry state, so **named classes are the default**. `@FunctionalInterface` makes the compiler enforce "exactly one abstract method".

Exercise 16 — Cache eviction policies

A fixed-size cache evicts one entry when full. Policies: LRU, FIFO, RANDOM, SIZE_AWARE (never evict entries smaller than N bytes).

Your main mistake: the strategy was doing the cache's job.

```
// x your version
interface EvictionPolicyStrategy {
    void put(int key, int value, Map<Int,String> mp, int size);
}
```

- The policy now owns **insertion**. "Check full, evict, insert" gets duplicated in all four classes.
- You hand it your internal `Map` — it can clear it, insert garbage, remove ten entries. The cache lost control of its own state.
- `Cache.put()` no longer puts anything.

A strategy **DECIDES**, it doesn't **ACT**. Give it enough to make the decision; let the context carry it out. An eviction policy's one job: **name which key should go**.
Pass the minimum the decision needs, and if it must read more, hand it a read-only view: `Collections.unmodifiableMap(entries)`.

```
interface EvictionPolicy {
    Integer selectVictim(Map<Integer, String> entries);    // returns WHO to evict
}

public void put(int key, String value) {                // cache stays in charge
    if (entries.size() >= maxSize && !entries.containsKey(key)) {
        Integer victim = policy.selectVictim(Collections.unmodifiableMap(entries));
        entries.remove(victim);
    }
    entries.put(key, value);
}
```

Other exercise 16 mistakes: `SIZE_AWARE`'s threshold not handled — it's configuration, so it goes in the strategy's constructor: `new SizeAwareEvictionPolicy(1024)`.
Class names `LRU`, `SIZE_AWARE` look like constants → `LruEvictionPolicy`, `SizeAwareEvictionPolicy` (PascalCase nouns). Don't stack words: `EvictionPolicy`, not `EvictionPolicyStrategy`.
`Map<Int,String>` → `Integer` (third time). Field `maxSize` vs `this.size` mismatch. `Main` vs `main`.
✓ Strategy structure, delegation, runtime swap, no if/else in the context — all correct.

Part B — "LRU needs to know the order of access"

Your doubt: "what does it mean that `get()` has to tell the strategy a key was touched?" Trace it, capacity 3:

```
cache.put(1, "a"); cache.put(2, "b"); cache.put(3, "c");
cache.get(1);      // read key 1 – nothing added or removed
cache.put(4, "d");  // FULL – who is evicted?

FIFO → evicts by INSERTION order → key 1 went in first → evicts 1 (the read is irrelevant)
LRU  → evicts the LEAST RECENTLY USED; reading counts as use → key 1 was just used → evicts 2
```

Same calls, different victims — only because `get(1)` happened. But a plain `get()` tells nobody, and a `Map` has no notion of access order, so LRU would be guessing. **So `get()` must notify the policy.**

The design problem: if `onAccess()` goes on the interface, `FIFO` and `RANDOM` must implement it as empty bodies — the fat-interface / Liskov symptom.

The hint you skipped: `FIFO` has the same problem. A `HashMap` has no insertion order either, so `FIFO` also needs to be *told* — about insertions.

Two solutions

	Separate capability interface (yours)	Default methods (reference)
Honesty	The type declares exactly what it supports	Every policy "has" all hooks
Cache code	an <code>instanceof</code> check per hook	clean, unconditional calls
Scaling	3 hooks = 3 interfaces = 3 checks	add a hook, nothing else changes

Your Part B compile error (same as exercise 13):

```
if (this.policy instanceof UpdateAfterRead) { this.policy.onAccess(key); } // x
if (policy instanceof AccessAware a) { a.onAccess(key); }                // ✓
```

`instanceof` is one word, and **the declared type decides what's callable** — `policy` is an `EvictionPolicy`, which has no `onAccess()`. This `instanceof` is legitimate (asking "can it do X?"), because one field must hold all four policy types. Also: interface names are capitalised capabilities (`AccessAware`), not `updateAfterRead`.

Rule of thumb: separate interface when **callers choose based on the capability** (e.g. `Refundable` in a method signature); default methods when **one holder needs uniform hooks** (lifecycle callbacks). The JDK uses defaults exactly this way.

The complete reference solution

```

interface EvictionPolicy {
    /** Choose a key to evict. Must return a key in the cache, or null. Must NOT mutate. */
    Integer selectVictim(Map<Integer, String> entries);

    /** Optional notifications – policies that don't need them may ignore them. */
    default void onInsert(int key) { }
    default void onAccess(int key) { }
    default void onRemove(int key) { }
}

class LruEvictionPolicy implements EvictionPolicy {
    private final LinkedHashSet<Integer> usageOrder = new LinkedHashSet<>(); // first = least recent
    public Integer selectVictim(Map<Integer, String> entries) {
        return usageOrder.isEmpty() ? null : usageOrder.iterator().next();
    }
    @Override public void onInsert(int key) { touch(key); }
    @Override public void onAccess(int key) { touch(key); }
    @Override public void onRemove(int key) { usageOrder.remove(key); }
    private void touch(int key) { usageOrder.remove(key); usageOrder.add(key); } // move to end
}

class FifoEvictionPolicy implements EvictionPolicy {
    private final Queue<Integer> insertionOrder = new LinkedList<>();
    public Integer selectVictim(Map<Integer, String> entries) { return insertionOrder.peek(); }
    @Override public void onInsert(int key) { insertionOrder.offer(key); }
    @Override public void onRemove(int key) { insertionOrder.remove(key); }
    // onAccess NOT overridden – FIFO ignores reads. That's the whole point.
}

class RandomEvictionPolicy implements EvictionPolicy {
    private final Random random = new Random();
    public Integer selectVictim(Map<Integer, String> entries) {
        if (entries.isEmpty()) return null;
        List<Integer> keys = new ArrayList<>(entries.keySet());
        return keys.get(random.nextInt(keys.size()));
    }
}

class SizeAwareEvictionPolicy implements EvictionPolicy {
    private final int minBytes; // configuration, in the constructor
    public SizeAwareEvictionPolicy(int minBytes) { this.minBytes = minBytes; }
    public Integer selectVictim(Map<Integer, String> entries) {
        return entries.entrySet().stream()
            .filter(e -> e.getValue().length() >= minBytes)
            .max(Comparator.comparingInt(e -> e.getValue().length()))
            .map(Map.Entry::getKey)
            .orElse(null);
    }
}

class Cache {
    private final Map<Integer, String> entries = new HashMap<>();
    private final int maxSize;
    private EvictionPolicy policy;

    public Cache(EvictionPolicy policy, int maxSize) { this.policy = policy; this.maxSize = maxSize; }
    public void setEvictionPolicy(EvictionPolicy policy) { this.policy = policy; }

    public String get(int key) {
        String value = entries.get(key);
        if (value == null) throw new KeyNotFoundException(key);
        policy.onAccess(key); // no instanceof needed
        return value;
    }

    public void put(int key, String value) {
        if (entries.containsKey(key)) { // overwrite: no eviction, but it IS a use
            entries.put(key, value);
            policy.onAccess(key);
            return;
        }
        if (entries.size() >= maxSize) {
            Integer victim = policy.selectVictim(Collections.unmodifiableMap(entries));
            if (victim != null) { entries.remove(victim); policy.onRemove(victim); }
        }
        entries.put(key, value);
        policy.onInsert(key);
    }
}

```

- **The cache knows nothing about policies** — no instanceof, no if/else, no names.
- **Each policy keeps its own bookkeeping** (LinkedHashSet / Queue / nothing).
- **onRemove prevents a real bug**: without it, LRU's set fills with dead keys and can nominate an already-evicted key.
- **Runtime-swap catch**: a policy swapped in mid-flight has empty bookkeeping. Options: seed it with current keys, accept brief degradation, or add an `initialise(keys)` hook. Naming this unprompted is a strong signal.

Your doubt: "FIFO doesn't define onAccess — what happens when the cache calls it?" Java looks for `onAccess` on the actual object (`FifoEvictionPolicy`), doesn't find it, walks up to the interface, finds the `default` body, runs it (empty), continues. **No error, no crash, nothing happens — exactly what FIFO wants.** Without `default` the method would be abstract and FIFO wouldn't compile. An override always wins over a default. To call a default explicitly: `EvictionPolicy.super.onAccess(key);`

Your doubt: "an empty default body — isn't that a Liskov violation?" It depends entirely on **what the contract promises**.
`Logger.log(msg)` promises "the message is recorded" → an empty body breaks that promise → **violation**.
`onAccess(key)` promises only "you are being informed" → doing nothing with the information honours it → **fine**.
The test: does the caller rely on an observable effect of the call? The cache doesn't — it never checks what the policy did. The real promise is `selectVictim()`, and every policy keeps it.

A default body is fine for...	A default body is a violation for...
optional lifecycle hooks / notifications	core operations callers depend on (<code>refund</code> , <code>pay</code> , <code>log</code>)
"doing nothing" is a valid response	doing nothing means the caller was lied to

Document the intent in Javadoc ("Optional notification — may be ignored") so an empty implementation provably honours the contract.

Part C answers

Q6 — "we only ever use LRU": YAGNI + KISS. With one policy, inline it into the cache (a single-implementation interface is indirection). Staged answer: "If eviction logic grew complex enough to need its own tests, I'd extract a class first, and add the interface only when a second policy appeared."

Q7 — which policies work badly as lambdas? Three of four. `RANDOM` works (stateless). `SIZE_AWARE` needs `minBytes` config. `LRU` and `FIFO` need bookkeeping **and** override default hooks — a lambda supplies only the one abstract method, so it can never override a default.

2. Factory

The problem

Strategy leaves a question open: somewhere a concrete class still has to be created.

```
class OrderService {
    public void checkout(String type, double amount) {
        PaymentMethod method;
        if (type.equals("UPI")) method = new UpiPayment("ram@bank");
        else if (type.equals("CARD")) method = new CreditCardPayment("1234");
        else method = new CashPayment();
        method.pay(amount);
    }
}
```

1. **DIP violation** — business logic names concrete classes and knows their construction details.
2. **OCP violation** — a new payment type edits this method.
3. **It spreads** — the same if/else gets copied into refunds, reports, everywhere.

The fix: move object creation out of business logic into one dedicated place. A factory hides **which class** AND **how to build it** (page size, DPI, delimiter...).

Variant 1 — Simple Factory (not GoF, but the most used)

```
class PaymentMethodFactory {
    public static PaymentMethod create(String type) {
        switch (type) {
            case "UPI": return new UpiPayment("ram@bank");
            case "CARD": return new CreditCardPayment("1234");
            case "CASH": return new CashPayment();
            default: throw new UnsupportedOperationException(type);
        }
    }
}

PaymentMethod method = PaymentMethodFactory.create(type);    // no concrete names in business logic
```

"You've just moved the if/else." Yes, deliberately. The switch still exists, but in **exactly one known place** instead of scattered through business logic, and the business logic now depends only on the interface.

Variant 2 — Registry Factory (no switch at all)

```
class PaymentMethodFactory {
    private final Map<String, Supplier<PaymentMethod>> registry = new HashMap<>();

    public void register(String type, Supplier<PaymentMethod> creator) { registry.put(type, creator); }

    public PaymentMethod create(String type) {
        Supplier<PaymentMethod> creator = registry.get(type);
        if (creator == null) throw new UnsupportedOperationException(type);
        return creator.get();    // runs the recipe → a fresh object
    }
}

factory.register("UPI", () -> new UpiPayment("ram@bank"));
factory.register("CASH", CashPayment::new);    // constructor reference = () -> new CashPayment()
```

Adding a type = one registration line, and the factory class itself never changes. Registrations can come from config, a Spring context, even a plugin loaded at runtime — a switch requires every type at compile time.

Your doubt: "why Supplier<PaymentMethod> and not PaymentMethod? And what does creator.get() do?"

```
// Option 1 – store the OBJECT: created once, the same object returned every time
Map<String, PaymentMethod> registry = new HashMap<>();
registry.put("UPI", new UpiPayment("ram@bank"));    // object built HERE, once
return registry.get(type);    // same object, every caller

// Option 2 – store a RECIPE: nothing is built until you ask
Map<String, Supplier<PaymentMethod>> registry = new HashMap<>();
registry.put("UPI", () -> new UpiPayment("ram@bank"));    // nothing created yet!
return registry.get(type).get();    // NOW the lambda runs → new object

interface Supplier<T> { T get(); }    // just a single-method interface
// registry.get(type) → Map.get: fetch the recipe
// registry.get(type).get() → Supplier.get: run the recipe (same name, different method)
```

Why it matters — the shared-object bug:

```

class UpiPayment implements PaymentMethod {
    private final String upiId;
    private String status = "PENDING";
    public void pay(double amount) { this.status = "COMPLETED"; }    // mutates itself
}

// SHARED instance
PaymentMethod a = factory.create("UPI");  a.pay(500);    // a.status = COMPLETED
PaymentMethod b = factory.create("UPI");    // the SAME object
b.getStatus();                                // "COMPLETED" – b never paid!

// FRESH instance (Supplier)
PaymentMethod b = factory.create("UPI");    // brand new object
b.getStatus();                                // "PENDING" – correct

```

Your follow-up: "so each get() creates a new object, so different threads get their own?" Yes. Precisely: fresh objects help **because each use gets its own state**, not because of threads. Two sequential calls in one thread already show the bug. Threads make it worse — two threads mutating one object *simultaneously* give corrupted states, not just stale ones.

Configuration vs state — what actually decides "shareable"

Your doubt: "EqualSplit takes no constructor argument, so no new state is created?" Usually true, but it's not the rule. Constructor arguments are irrelevant; what matters is **whether any field is written after construction**.

```

class PercentageSplit implements SplitStrategy {
    private final List<Double> percentages;    // CONFIG: set once, final, only read
    ...                                       // → safe to share, despite the argument
}

class UpiPayment {
    private final String upiId;                // config – fine
    private String status = "PENDING";        // STATE – written by pay() → don't share
}

```

	Example	Share?
Configuration — set in constructor, final, only read	<code>percentages</code> , <code>minBytes</code> , <code>upiId</code> , <code>delimiter</code>	✓ yes
State — written after construction	<code>status</code> , <code>usageOrder</code> , <code>counter</code> , <code>pageBuffer</code>	✗ no

Quick check: scan for writes outside the constructor. `this.status = ...` or `list.add(...)` → stateful → fresh per use. Only reads and locals → share it.
Trap: `final` on a collection field only stops *reassignment* of the reference; its *contents* can still change. `private final Set<Integer> usageOrder` is still state.
One sentence: stateless (immutable) objects can be shared; stateful ones cannot. Spring's version of this decision: `@Scope("singleton")` vs `@Scope("prototype")`.

Variant 3 — Factory Method (the actual GoF pattern)

```

abstract class RideService {
    protected abstract Vehicle createVehicle();    // the factory method

    public final void startRide(String from, String to) {
        Vehicle vehicle = createVehicle();    // subclass decides WHICH
        vehicle.dispatch(from);
    }
}

class AutoRideService extends RideService { protected Vehicle createVehicle() { return new Auto(); } }
class CabRideService extends RideService { protected Vehicle createVehicle() { return new Cab(); } }

```

Simple Factory picks by a **parameter**; Factory Method picks by **which subclass you're in**. It's Template Method where the deferred step is object creation.

Variant 4 — Abstract Factory (families that must match)

```

interface ExporterFactory {
    Exporter createPdfExporter();
    Exporter createExcelExporter();
}

class PremiumExporterFactory implements ExporterFactory {
    public Exporter createPdfExporter() { return new PdfExporter(PageSize.A4, 600, "CONFIDENTIAL"); }
    public Exporter createExcelExporter() { return new ExcelExporter("Sheet1", BRANDED_TEMPLATE); }
}

class FreeExporterFactory implements ExporterFactory {
    public Exporter createPdfExporter() { return new PdfExporter(PageSize.A4, 150, null); }
    public Exporter createExcelExporter() { return new ExcelExporter("Sheet1", null); }
}

```

The phrase: "Abstract Factory makes mismatched combinations **impossible by construction**." One object produces the whole set, so a premium PDF alongside a plain Excel can't happen. Use it only when objects must vary **together**.

When NOT to use a factory

- One implementation, simple construction — `new ArrayList<>()` needs no factory.
- The caller legitimately knows which type it wants.

- The factory would do nothing but call `new` — ceremony (YAGNI).

A factory earns its place when the choice of type depends on runtime data (a string, config, user input) and is needed in more than one place.

JDK/Spring: `Calendar.getInstance()`, `NumberFormat.getInstance(locale)`, `Executors.newFixedThreadPool(n)`, `Integer.valueOf` (caches `-128..127` — why `Integer a = 127, b = 127; a == b` is true but fails at 1000), Spring's `BeanFactory`, `@Bean` methods.

	Factory	Builder (pattern 4)
Answers	Which type do I create?	How do I assemble one complex object?
Varies across	different classes	many optional parameters

Static factory method idiom $\neq$ Factory pattern. `Report.empty()`, `Report.fromCsv(file)`, `Integer.valueOf(127)` are named alternatives to a constructor **on the same class** (Bloch, Item 1). They earn their place by adding a name, caching, or validation that `new` can't. The Factory pattern is a separate class choosing between types.

Exercise 17 — Report exporters

Part A — simple factory

Diagnosed the problem, didn't apply the fix (3rd time): you correctly said `new EmailSender()` should be injected, added the constructor... then still called `new EmailSender().send(...)` inside the method, leaving the injected field unused. Same with `new ExportFactoryService()`. **Re-read your own critique and check each fix actually appears in the code.** The interviewer grades the code, not the analysis.

The problem you missed — what ReportService was forced to KNOW: `new PdfExporter(A4, 300)`, `new ExcelExporter("Sheet1")`, `new CsvExporter(',',')`. A service coordinating export + email held knowledge about DPI and delimiters. That's a leaky abstraction — separate from DIP and OCP. The factory should hold those defaults: `case "PDF": return new PdfExporter(PageSize.A4, 300);`

Wrong label: branching on format is **OCP** (a new format reopens the method). The **SRP** problem is separate: exporting AND emailing are two actors. Name the right principle for the right symptom.

Switch bugs:

```
public Exporter createExporter(Report type) {          // x should be String type
    switch (type) {
        case 'PDF':                                  // x single quotes = one char; Strings use "PDF"
            ...
        Default: throw RuntimeException("wrong type"); // xx see below
    }
}
```

Default: (capital D) is not the keyword — it's a **label**, which compiles silently and never runs. Must be `default: . throw` needs `new`. Prefer a domain exception: `throw new UnsupportedOperationException(format);`

Naming: interface `exporter` → `Exporter`; `ExportFactoryService` → `ExporterFactory` (a factory, not a service); variable `exportFormat` holds an `exporter` → `exporter`.

✓ Good reasoning for choosing simple over registry: a small fixed set of formats, no runtime registration needed.

```
class ExporterFactory {
    public Exporter create(String format) {
        switch (format) {
            case "PDF":    return new PdfExporter(PageSize.A4, 300); // config lives HERE
            case "EXCEL":  return new ExcelExporter("Sheet1");
            case "CSV":    return new CsvExporter(',',');
            default:       throw new UnsupportedOperationException(format);
        }
    }
}

class ReportService {
    private final ExporterFactory factory;
    private final EmailSender sender;
    public ReportService(ExporterFactory factory, EmailSender sender) {
        this.factory = factory; this.sender = sender;
    }
    public void export(String format, Report report, String email) {
        Exporter exporter = factory.create(format); // no format comparison, no concrete names
        exporter.export(report);
        sender.send(email, "Report attached");      // uses the INJECTED sender
    }
}
```

Static vs instance factory: static is simple but can't be injected or faked; an injected instance is testable and consistent when the service already receives dependencies.

Part B — premium vs free families (Abstract Factory)

1. You built a third class AbstractUserExportFactory that switched on userType. The name misled you: **"Abstract" refers to the interface**, which you already had. The two tier factories ARE the pattern. The extra class reintroduced the switch in every method (OCP back again). Select the family **once, at the edge**:

```
ExporterFactory factory = isPremium ? new PremiumExporterFactory() : new FreeExporterFactory();
ReportService service = new ReportService(factory, sender);
```

2. Signature drift. Interface declared `pdfExport(Report)`, implementations declared `pdfExport(Report, PageSize)` — different methods, so the class wouldn't implement the interface. Later, two methods were both named `excelExport`. **Write the interface once, then copy-paste each signature into implementations. Never retype.**

3. The factory was doing the exporting (methods returned `void` and exported internally). **A factory creates and hands back; it doesn't use.** Return type `Exporter`, and let the service call `export()`. Same rule as "a strategy decides, it doesn't act".

4. exportWithWatermark() broke the product interface. A caller holding an `Exporter` can't reach it (declared type again). **Put variation into construction:** `new PdfExporter(A4, 600, "CONFIDENTIAL")` vs `new PdfExporter(A4, 150, null)`. Then both are plain `Exporter`s and the interface stays uniform.

✓ **You got the core insight:** one factory per family, sharing an interface. That's the part most people miss.

Part C

Q6 — ReportFactory.createReport() { return new Report(); }: remove it. One class, no decision, nothing to hide — ceremony (YAGNI, KISS). Just `new Report()`. A static factory method is only worth it when it adds a meaningful name, caching or validation.

Your doubt on Q7: "I call new PdfExporter every time — is it still shared?" **No.** `new` on every call means every caller gets its own object; your code was correct. The question asked something else: *should* it be fresh or cached, and **what information decides that?**

```
// Stateless — only final config, a LOCAL writer inside export(): sharing is safe AND better
class CsvExporter implements Exporter {
    private final char delimiter;
    public void export(Report r) { CsvWriter w = new CsvWriter(delimiter); w.write(r); }
}
// Stateful — export() writes into fields: a shared instance corrupts the next report
class PdfExporter implements Exporter {
    private int currentPage = 0;
    private List<String> pageBuffer = new ArrayList<>();
    public void export(Report r) { for (...) { currentPage++; pageBuffer.add(render(s)); } ... }
}
// shared PdfExporter: sales report (5 pages) then HR report (3 pages)
// → the HR file contains 8 pages: the sales report AND the HR report
// with threads: pages from two reports interleave; ArrayList itself can corrupt

class ExporterFactory {
    // same factory, per-type decision
    private final Exporter csvExporter = new CsvExporter(','); // stateless → cached
    public Exporter create(String format) {
        switch (format) {
            case "PDF": return new PdfExporter(PageSize.A4, 300); // stateful → fresh
            case "CSV": return csvExporter; // shared
            ...
        }
    }
}
```

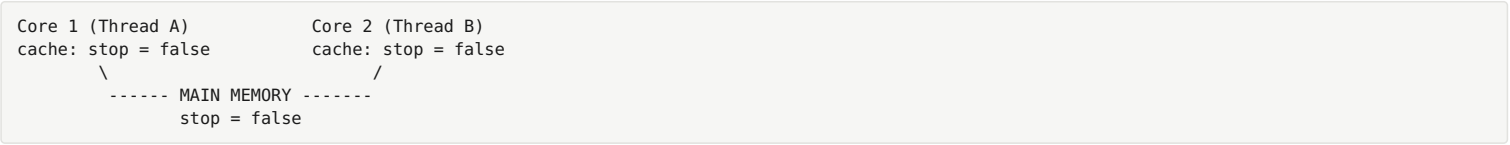
The Q7 answer: "Fresh per call is the safe default. If an exporter is stateless — only final config, no fields written during export — I'd cache one instance, since allocating identical objects is waste. If it accumulates per-export state like a page buffer, it must be fresh, especially with concurrent requests. I'd check the class before deciding." **The factory controls object lifetime — that's a decision, not an accident.**

3. Concurrency Foundations

You need these to understand Singleton — and they come back in every booking, parking and ride problem in Phase 5.

How threads actually read and write

Your doubt: "I want to understand how threads work while reading and writing." The wrong mental model is "all threads read and write one shared memory, instantly." In reality each CPU core has its own **cache**, and writes sync to main memory whenever the CPU decides.



This creates three separate hazards:

Hazard	What goes wrong	Example
Visibility	Thread B never sees Thread A's write — it's still sitting in A's cache	stop = true in A; B's while (!stop) loops forever
Reordering	The compiler/CPU runs instructions out of order (legal if the result looks the same <i>to that one thread</i>); another thread sees a strange intermediate state	instance = new X() — reference assigned before the constructor finishes
Atomicity	An operation that looks like one step is several; threads interleave between the steps and lose updates	counter++

counter++ is three steps

1. read counter (5)
2. add 1 (6)
3. write counter back (6)

Thread A: reads 5
Thread B: reads 5
Thread A: writes 6
Thread B: writes 6

← A hasn't written yet
→ two increments, counter only went up by ONE (lost update)

In a booking system that means two bookings get the **same ID**.

Check-then-act — the same bug in a different shape

if (!bookedSeats.contains("A1")) // CHECK
bookedSeats.add("A1"); // ACT

Thread A: contains("A1")? → false
Thread B: contains("A1")? → false
Thread A: add → "booked"
Thread B: add → "booked"

← A hasn't added yet
→ two customers hold seat A1. No exception, no log.

Name the SHAPE, not just "it's not atomic": read-modify-write (counter++) and check-then-act (if (x == null) x = ..., if (!contains) add, if (!isEmpty) remove). Interviewers want the trace, not "it gets corrupted".

volatile, in plain terms

A `volatile` field tells the JVM two things:

- "Never cache this. Always read from and write to main memory." → fixes visibility.
- "Don't reorder instructions across this read/write." → fixes reordering.

```
private volatile boolean stop = false; // shutdown() is now seen by the worker immediately
```

What volatile does NOT do: atomicity.

```
private volatile int counter = 0;
counter++; // STILL BROKEN
// A reads a fresh 5, B reads a fresh 5, both write a fresh 6 – update still lost
```

volatile on a collection reference protects the reference, not the contents.

```
private volatile Set<String> bookedSeats = new HashSet<>();
bookedSeats = new HashSet<>(); // ← volatile protects THIS (reassignment) – which you never do
bookedSeats.add("A1"); // ← volatile does NOTHING here (mutation) – which you do constantly
```

Interview line: "volatile on a collection reference guards reassignment, not mutation, so it buys me nothing here."

volatile vs synchronized vs Atomic

	volatile	synchronized	AtomicInteger etc.
Visibility	✓	✓	✓
Prevents reordering	✓	✓	✓
Atomic read-modify-write	✗	✓	✓
Blocks other threads	no	yes	no (CPU compare-and-swap)
Use for	a flag one thread sets; a reference assigned once then read	multi-step critical sections	a single counter

The one-line rule: **volatile** = "everyone sees the latest value". **synchronized** = "only one thread does this at a time". Only setting or only reading → volatile is enough. Read, compute, write back → synchronized or an Atomic class.

Don't stack them: volatile + synchronized on the same state (your exercise 18) suggests you're unsure which one solves what — synchronized already includes visibility.

```
private final AtomicInteger counter = new AtomicInteger();
counter.getAndIncrement();           // read-add-write as ONE uninterruptible step
counter.updateAndGet(c -> Math.max(0, c - 1)); // "decrement but never below 0" – atomic
```

Class initialisation — the fact Singleton is built on

Your doubts: "static fields are initialised only once?" · "static methods are not one-time?" · "static classes are also initialised once?" Pin down the words: **fields are initialised; methods are not initialised at all — they just run.**

```
class Demo {
    static int field = init("FIELD");           // static field initialiser
    int instanceField = init("INSTANCE");       // instance field initialiser
    static int init(String s) { System.out.println(s + " ran"); return 0; }
    static int method() { return init("METHOD"); }
}
new Demo(); new Demo(); Demo.method(); Demo.method();

// Output:
// FIELD ran           ← once, when the class first loads
// INSTANCE ran        ← per new
// INSTANCE ran
// METHOD ran           ← every call
// METHOD ran
```

Thing	Runs how often	Thread-safe by JVM?
static int x = ...; / static { ... }	once, at class initialisation	✓
int y = ...; (instance field)	once per new	✗
static void m() { ... }	every call	✗
void m() { ... }	every call	✗

Initialisers run once. Method bodies run every time. static on a method only means "belongs to the class, call it as `ClassName.m()`, no `this`" — it says nothing about how often it runs or thread safety.

- A class is loaded and initialised on its **first reference** (static method call, static field read, `new`) — not at program start. Never referenced → never initialised.
- The JVM initialises each class **exactly once** and holds an internal lock while doing it; concurrent threads block until it's done.
- This is true of **all** classes. `static` on a *nested* class is a separate concern: it means the class doesn't need an enclosing instance.
- Precisely, it's once **per classloader** — which is why multiple classloaders can break a singleton.

```
class Outer {
    static class A { static int x = print("A initialised"); }
    static class B { static int x = print("B initialised"); }
}
Outer.A.x; Outer.A.x;           // prints "A initialised" ONCE; B never prints – never referenced
```

Your doubt: "if two threads call new ClassName(), only one can create an object at a time? That seems conflicting." Right to challenge — that was imprecise. **Object creation (new) is never locked.** Ten threads calling `new ArrayList<>()` get ten lists concurrently. What's locked is **class initialisation**, a one-time event.

Event	Happens	Locked?
Class initialisation	once ever, first time the class is touched	✓ by the JVM
Object creation (new)	every time new runs	✗ fully concurrent

In Bill Pugh, `new` sits inside a **static field initialiser**, so it runs as part of initialising the `Holder` class — and that's what the JVM protects. **The new was incidental.** In the broken lazy version, `new` sits in an ordinary method body, so nothing protects it.

Locks: every object has one

```
public synchronized void book(String seatId) { ... }
// is exactly the same as
public void book(String seatId) { synchronized (this) { ... } } // locks THIS object
// static synchronized → locks the Class object (e.g. BookingIdGenerator.class)
```

- Every Java object has a built-in lock (its *monitor*). `synchronized (x)` acquires **x's** lock.
- **A lock doesn't freeze the object.** It only blocks other threads trying to take the *same lock*. Code that doesn't use `synchronized` walks straight in. So **every** method touching shared state must use the same lock — including reads, or a reader can see a half-finished write.
- Locks are an agreement between pieces of code, not a wall around the data.

Lock granularity — coarse vs per-key

Your doubt: "here we locked that whole object? Object lock = seatLocks.computeIfAbsent(seatId, k -> new Object());" No — it locks a **tiny separate object per seat**. `new Object()` holds no data; it exists only to have its own lock, like a key per room.

```
private final ConcurrentHashMap<String, Object> seatLocks = new ConcurrentHashMap<>();
public void book(String seatId) {
    Object lock = seatLocks.computeIfAbsent(seatId, k -> new Object());
    synchronized (lock) { /* multi-step booking for THIS seat */ }
}

seatLocks: "A1" → Object@11, "B7" → Object@42
Thread A: book("A1") → locks Object@11 → working
Thread B: book("B7") → locks Object@42 → runs IN PARALLEL
Thread C: book("A1") → wants Object@11 → WAITS for A      (only same-seat threads queue)
```

Why computeIfAbsent : a naive get-then-put lets two threads each create their own lock object for A1 — they'd lock different objects and both book it. computeIfAbsent is atomic, so one lock object per key, ever. (With millions of keys, use **lock striping**: a fixed pool of locks chosen by hash.)

Analogy: coarse synchronized = one key for the whole building. Per-seat locks = one key per room; people only queue for the same room.

ConcurrentHashMap

Your doubt: "I have never used ConcurrentHashMap, explain more."

How it's thread-safe

HashMap	not at all — concurrent puts can lose entries or corrupt buckets during resize
Collections.synchronizedMap(map)	one lock on the whole map for every call (coarse — only one thread at a time, even reads)
ConcurrentHashMap	fine-grained internal locking ($\approx$ per bucket); most reads don't lock; different keys rarely block each other

The trap: each call is safe; a SEQUENCE of calls is not.

```
if (!map.containsKey("A1")) {    // safe on its own
    map.put("A1", "Ram");        // safe on its own
}                                // x another thread can run BETWEEN the lines → check-then-act
```

A thread-safe collection doesn't close a gap between *your* calls. Using ConcurrentHashMap alone would NOT fix double-booking.

The atomic compound methods — use these instead

```
map.putIfAbsent("A1", "Ram");           // null if it succeeded, else the existing value
map.computeIfAbsent(k, key -> create(key)); // create once per key, return it
map.merge("apple", 1, Integer::sum);     // absent → 1, present → old + 1 (atomic counter)
map.compute(k, (key, old) -> old == null ? 1 : old + 1);
map.remove("A1", "Ram");                // remove only if currently mapped to "Ram"

Set<String> booked = ConcurrentHashMap.newKeySet(); // there is no ConcurrentHashMap class
if (!booked.add("A1")) throw new SeatAlreadyBookedException("A1"); // add returns false if present
```

Use the method's boolean/return value instead of checking first. Set.add() and Set.remove() return whether they did anything, so one call both checks and acts. Your cancel() used if (!bookedSeats.remove(id)) throw ... — exactly right.

- No null keys or values (a null from get would be ambiguous under concurrency).
- size() is approximate while others are writing.
- Iteration never throws ConcurrentModificationException; it may or may not see concurrent changes.
- Read without creating: map.get(zone) (or getOrDefault) — not computeIfAbsent, or the map fills with keys that were only queried.

Other concurrent tools met so far: DoubleAdder (thread-safe accumulator, add()/sum()), BlockingQueue/ArrayBlockingQueue (thread-safe poll()/offer(), plus take() which waits).

4. Singleton

Intent: exactly one instance of a class exists, with one way to reach it. Interviewers ask it to test **thread safety** and whether you know **when not to use it**; the basic version alone scores nothing.

Every singleton needs BOTH: (1) a **private constructor** — so nobody outside can write `new`; (2) a **static accessor** — the only way in. Miss either and it isn't a singleton. Without any declared constructor, Java inserts a default **public** one.

Version 1 — Eager

```
public class ConfigManager {
    private static final ConfigManager INSTANCE = new ConfigManager();    // built at class init

    private final Map<String, String> settings = new HashMap<>();

    private ConfigManager() {
        settings.put("db.url", "localhost:5432");
    }

    public static ConfigManager getInstance() {
        return INSTANCE;          // NOT this.INSTANCE — static methods have no "this"
    }

    public String get(String key) { return settings.get(key); }
}
```

Thread-safe for free — static field initialisers run once under the JVM's class-init lock. **Trade-off:** built at class load whether or not you ever call `getInstance()`.

Your doubt: "what does 'construction is cheap' mean?" It's about what the constructor does. **Cheap:** field assignments, an empty collection — microseconds. **Expensive:** opening DB connections (network, ~100 ms each), reading a 200 MB file, loading a big cache into RAM (memory counts too). Eager runs the constructor at class load even in runs that never use the instance. **Eager when construction is cheap AND the instance is almost certainly used; lazy (Bill Pugh) when it's expensive or may be unused.** `SeatRegistry` = cheap → eager is fine. `ConnectionPool` = expensive → Bill Pugh.

Version 2 — Lazy (BROKEN — know why)

```
public class ConfigManager {
    private static ConfigManager instance;
    private ConfigManager() { }

    public static ConfigManager getInstance() {
        if (instance == null) {
            instance = new ConfigManager();    // two threads can both be here
            // both create an object
        }
        return instance;
    }
}
// A: instance == null → true; B: instance == null → true; A creates #1; B creates #2
```

Check-then-act in an ordinary method body — nothing protects it. **The version interviewers show you and ask "what's wrong?"**

Version 3 — synchronized (correct, slow)

```
public static synchronized ConfigManager getInstance() {
    if (instance == null) instance = new ConfigManager();
    return instance;
}
```

Every call takes a lock forever, though the race only exists on the first call.

Version 4 — Double-checked locking

```
public class ConfigManager {
    private static volatile ConfigManager instance;    // volatile is ESSENTIAL
    private ConfigManager() { }

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) {
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

- ① **Outer check** — once the instance exists, return immediately with no locking.
- ② **Inner check** — two threads can both pass ① before either takes the lock; the second must re-verify.
- **Why volatile** — `instance = new X()` is three steps: allocate, run constructor, assign reference. Steps 2 and 3 may be **reordered**:

```
Thread A: allocates memory
Thread A: assigns instance = <address>      ← reordered: before the constructor ran
Thread B: ① instance == null → FALSE → returns it, skips the lock entirely
Thread B: uses the object → fields still null/0 → garbage or NullPointerException
Thread A: runs the constructor              ← too late
```

`volatile` forbids that reordering and makes the write visible. **Without it, DCL is silently broken.** (Here `volatile` is needed for ordering + visibility; atomicity comes from the synchronized block.)

```
// Working timeline
A: ① null → true          B: ① null → true
A: ② takes the lock       B: ② BLOCKS
A: ③ null → true, ④ creates (constructor finishes before the reference is visible), releases
                        B: ② takes the lock, ③ null → FALSE, skips, releases
both return the SAME instance
```

Version 5 — Bill Pugh holder (best plain-Java option)

```
public class ConfigManager {
    private final Map<String, String> settings = new HashMap<>();

    private ConfigManager() {                // private – half the pattern
        settings.put("db.url", "localhost:5432");
    }

    private static class Holder {            // static – REQUIRED
        private static final ConfigManager INSTANCE = new ConfigManager(); // a FIELD initialiser
    }

    public static ConfigManager getInstance() { // static + return type
        return Holder.INSTANCE;               // only READS a field – no "new" here
    }

    public String get(String key) { return settings.get(key); }
}
```

Your doubt: "explain the inner class approach again." **Laziness:** a class loads on first reference. `Holder` is referenced *only* inside `getInstance()`. Loading `ConfigManager` (e.g. calling some other static method) doesn't touch `Holder`, so no instance is built until `getInstance()` actually runs.

Thread safety: class initialisation is done once under the JVM's lock, so ten simultaneous first callers: one runs the initialiser, nine block inside the JVM, all ten get the same fully-built object. **The locking exists — you just didn't write it.**

No ongoing cost: after init, `Holder.INSTANCE` is a plain static field read.

```
ConfigManager.version(); // loads ConfigManager; Holder NOT referenced → nothing built
ConfigManager.getInstance(); // reads Holder.INSTANCE → Holder loads NOW → "expensive setup"
ConfigManager.getInstance(); // Holder already initialised → same object, nothing printed
```

Your exercise 18 mistake: Holder written as a METHOD.

```
private static class Holder {
    getInstance() { return new BookingIdGenerator(); } // x runs on EVERY call → a new object each time
}
private static class Holder {
    private static final BookingIdGenerator INSTANCE = new BookingIdGenerator(); // ✓ runs ONCE
}
```

Signature of a correct Bill Pugh: there is no `new` inside `getInstance()`.

Your exercise 19 mistakes on the same code (from memory): `private class Holder` → must be `private static class` (a non-static inner class needs an enclosing instance, and can't even hold static fields) · `public getInstance()` → `public static SeatRegistry getInstance()` (missing return type AND static) · `holder.instance` → `Holder.INSTANCE` (class name; static final = UPPER_CASE) · `SeatRegistry() { }` → `private` · `synchronize` → `synchronized`, and modifiers go before the return type: `public synchronized void book(...)`.

Version 6 — Enum

```
public enum ConfigManager {
    INSTANCE;

    private final Map<String, String> settings = new HashMap<>();

    ConfigManager() { // implicitly private
        System.out.println("constructor running");
        settings.put("db.url", "localhost:5432");
    }

    public String get(String key) { return settings.get(key); }
}
ConfigManager a = ConfigManager.INSTANCE;
ConfigManager b = ConfigManager.INSTANCE;
System.out.println(a == b); // true – "constructor running" printed ONCE
```

Your doubt: "is an enum's object initialised only once?" Yes. An enum constant is really a `public static final` field of the enum type, created during class initialisation — once, thread-safely. You can't create a second one: enum constructors are implicitly private and `new ConfigManager()` is a compile error.

Joshua Bloch's recommendation — the **only** version immune to reflection and serialization attacks. Can't extend a class; less control over timing.

Which to use

Situation	Version
Cheap construction, almost always used	Eager — simplest thing that works
Expensive construction or may be unused	Bill Pugh holder
Maximum safety and simplicity	Enum
Asked to demonstrate memory-model knowledge	DCL with volatile — explain the reordering

Versions 2 and 3 exist only to be critiqued. DCL is hand-building what the JVM gives Bill Pugh for free — explain DCL, write Bill Pugh.

Reflection, in plain terms

Your doubt: "explain reflection easily." Normal code uses names the compiler checks. Reflection uses names as **strings, looked up while the program runs**. Analogy: normal code = calling a friend whose number you memorised; reflection = being handed a name, looking it up in a directory, and dialling whatever you find.

```
e.getName(); // normal: typo won't compile

Class<?> clazz = Class.forName("Employee"); // find a class BY NAME
Object obj = clazz.getDeclaredConstructor().newInstance();
Method m = clazz.getMethod("getName"); // find a method BY NAME
Object result = m.invoke(obj); // typo "getNmae" compiles, crashes at runtime
```

Why it exists: frameworks are compiled before your classes exist. Spring has never heard of your `OrderService`, so it scans your classes at startup and creates/wires them reflectively. JUnit finding `@Test`, Jackson reading fields for JSON, Hibernate mapping `@Entity` — all reflection. **Downsides:** no compile-time safety, slower, breaks encapsulation, IDE renames don't update the strings.

Breaking a singleton — and the defences

```
// 1. Reflection – walks past the private constructor
Constructor<ConfigManager> c = ConfigManager.class.getDeclaredConstructor();
c.setAccessible(true); // switches OFF the access check
ConfigManager second = c.newInstance(); // a second instance exists
// Defence:
private ConfigManager() {
    if (Holder.INSTANCE != null) throw new IllegalStateException("Already instantiated");
}

// 2. Serialization – deserialising builds a brand-new object
// Defence: Java calls readResolve() after deserialising; return the real one
private Object readResolve() { return getInstance(); }

// 3. Cloning – Defence:
@Override protected Object clone() throws CloneNotSupportedException {
    throw new CloneNotSupportedException();
}

// 4. Multiple classloaders – each loads its own copy (rarely defended)
```

"Can you break my singleton?" Reflection can call the private constructor via `setAccessible(true)`, and deserialisation creates a new instance — defences are a guard in the constructor and `readResolve()`. An enum singleton is immune to both, because the JVM forbids reflective creation of enum constants and serialises enums by name, looking up the existing constant.

Why Singleton is called an anti-pattern

```
class SomeService { // Option A – global access (the smell)
    public void doWork() {
        Connection c = ConnectionPool.getInstance().getConnection(); // buried in a method
    }
}

class SomeService { // Option B – injection
    private final ConnectionPool pool;
    public SomeService(ConnectionPool pool) { this.pool = pool; }
    public void doWork() { Connection c = pool.getConnection(); }
}
```

1. **Hidden dependency** — A's constructor says nothing about needing a pool.
2. **Untestable** — no seam to pass a fake; the test needs a live database.
3. **Violates DIP** — business logic names a concrete class.
4. **Global mutable state** — concurrency hazard; tests become order-dependent.

(An earlier explanation used a class called `BookingService` here. It was an invented example, not from your exercises — any consumer class works.)

Your doubt: "@Service beans are singleton-scoped but injected rather than fetched globally — didn't understand." Separate two things: **how many instances exist** and **how you get hold of one**. The classic singleton bundles "one instance" with "fetch it globally via `getInstance()`". Spring keeps the first and throws away the second:

```
@Service // Spring creates ONE ConfigManager for the whole app
public class ConfigManager { public String get(String key) { ... } }

@Service
public class OrderService {
    private final ConfigManager config;
    public OrderService(ConfigManager config) { this.config = config; } // Spring passes it in
}
// in a test:
new OrderService(new FakeConfigManager()); // impossible with getInstance()
```

	Classic singleton	Spring singleton bean
Instances	one	one
How code gets it	calls <code>getInstance()</code> globally	receives it in the constructor
Visible in the signature	✗	✓
Substitutable in tests	✗	✓
Boilerplate you write	private ctor, static accessor, thread-safety	none

The answer that scores: "I'd use a Singleton where exactly one instance is genuinely required — a connection pool, a config registry. But in application code I'd let the DI container manage a single instance and inject it, which gives the same guarantee without the global access point or the testing problem."

Singleton vs static utility class: a singleton is an *object* — it can implement an interface, be passed as a parameter, be substituted in tests, be lazy. Static utility for pure stateless functions (`Math` , `Collections`); singleton when you need one object with state and a lifecycle.

A thread-safe singleton is NOT a thread-safe object

Your doubt: "when one thread has created the object, is that object locked?" No. The JVM locks **class initialisation**, only while it runs. After that, the singleton is an ordinary object: any number of threads call its methods concurrently, with no lock anywhere.

```
public void set(String key, String value) {
    settings.put(key, value);    // 10 threads at once on a HashMap = corruption
}
// Fixes: ConcurrentHashMap, synchronized methods, or – cleanest – make it immutable
```

Interview line: "Bill Pugh makes **creation** thread-safe. It says nothing about the object's **methods** — those are a separate problem I handle inside the class."

Exercise 18 — BookingIdGenerator + ConnectionPool

The two independent bugs

```
public class BookingIdGenerator {
    private static BookingIdGenerator instance;
    private int counter = 0;
    private BookingIdGenerator() { }
    public static BookingIdGenerator getInstance() {
        if (instance == null) instance = new BookingIdGenerator();    // BUG 1: check-then-act → 2 instances
        return instance;
    }
    public int nextId() { return counter++; }                          // BUG 2: read-modify-write → lost update
}
```

Your doubt: "the static field runs once, so isn't this protected even if 2+ threads try?" The static field declaration `private static BookingIdGenerator instance;` runs once — and just sets `null`. That protection is useless. The `new` lives in `getInstance()`, an ordinary method body, which runs unprotected on every call. **Test: is the `new` on the right-hand side of a static field declaration (or in a `static {}` block)? If yes, JVM guarantees once. If it's inside a method body, you're on your own.**

```
public class BookingIdGenerator {
    private final AtomicInteger counter = new AtomicInteger();
    private BookingIdGenerator() { }
    private static class Holder {
        private static final BookingIdGenerator INSTANCE = new BookingIdGenerator();
    }
    public static BookingIdGenerator getInstance() { return Holder.INSTANCE; }
    public int nextId() { return counter.getAndIncrement(); }
}
```

The two fixes are independent. `Holder` guarantees one object — does nothing about the counter. `AtomicInteger` guarantees safe increments — does nothing about how many objects exist. Fix only one and you still get duplicate IDs. **Present Bill Pugh; mention DCL + volatile** to show you know the memory model.

ConnectionPool

Your doubt: "how can the Holder create 5 objects?" It doesn't. **The Holder creates ONE pool.** The five connections are created *inside* that pool, in its constructor. Two classes, two counts.

```
Holder.INSTANCE → ConnectionPool (one object)
    └ available: [conn-0, conn-1, conn-2, conn-3, conn-4]

private ConnectionPool() {
    // runs ONCE, when Holder initialises
    for (int i = 0; i < 5; i++) available.add(new Connection("conn-" + i));
}
```

Your mistake: `release()` inverted. You had `release()` *removing* and `getConnection()` calling `release()`. Think of the real thing: **borrowing takes a connection OFF the shelf; returning puts it BACK.** With your version, a client's `release(c)` removes nothing (`c` is already gone), so every connection is lost forever and the pool drains.

The guard in `release()` ("throw if pool is empty") was also backwards — an empty pool is the *normal* state when a connection comes back. Meaningful guards: `c != null`, and issued by this pool.

`available[0]` — `[]` on a `List`, sixth time → `available.remove(0)`.

```
public synchronized Connection getConnection() {
    if (available.isEmpty()) throw new NoConnectionAvailableException("All connections in use");
    return available.remove(0);    // remove(int) returns the removed element
}

public synchronized void release(Connection c) {
    if (!inUse.remove(c)) throw new IllegalArgumentException("Not issued by this pool");
    available.add(c);             // BACK on the shelf
}
```

Why synchronized, on BOTH methods: `isEmpty()` then `remove(0)` is check-then-act — with one connection left, two threads both see "not empty" and one crashes, or both get the same connection. Both methods lock the same object (`this`), so a `release` can't interleave with a `get` and corrupt the list.

```
// Lock-free alternative
private final BlockingQueue<Connection> available = new ArrayBlockingQueue<>(5);
public Connection getConnection() {
    Connection c = available.poll(); // thread-safe; null if empty
    if (c == null) throw new NoConnectionAvailableException("All in use");
    return c;
}
public void release(Connection c) { available.offer(c); }
// available.take() → WAITS until one is free
// available.poll(5, TimeUnit.SECONDS) → waits up to 5s, then gives up
```

Raise unprompted: "Should an exhausted pool fail fast or block the caller? I'd probably offer a timeout." Real pools do this.

Exercise 19 — SeatRegistry

Three concurrency bugs (the booking bug has two parts)

Where	Shape	What goes wrong
getInstance()	check-then-act	two registries → each thread's bookings invisible to the other
contains then add	check-then-act	two customers told they booked A1 — no exception
HashSet itself	unsafe structure	concurrent add can corrupt buckets / lose entries
totalBookings++	read-modify-write	two bookings, count +1

Note: the check-then-act bug survives even with a thread-safe collection — the gap is between your two calls.

Reference

```
public class SeatRegistry {
    private final Set<String> bookedSeats = new HashSet<>(); // final: reference never changes
    private int totalBookings = 0;

    private SeatRegistry() { }
    private static class Holder { private static final SeatRegistry INSTANCE = new SeatRegistry(); }
    public static SeatRegistry getInstance() { return Holder.INSTANCE; }

    public synchronized void book(String seatId) {
        if (bookedSeats.contains(seatId)) throw new SeatAlreadyBookedException(seatId);
        bookedSeats.add(seatId);
        totalBookings++;
    }
    public synchronized void cancel(String seatId) {
        if (!bookedSeats.remove(seatId)) throw new SeatNotBookedException(seatId); // one call: check+act
        totalBookings--;
    }
    public synchronized boolean isBooked(String seatId) { return bookedSeats.contains(seatId); }
    public synchronized int getTotalBookings() { return totalBookings; }
}
```

Your exercise 19 misses: cancel() did one of three requirements (free seat ✓, decrement ✗, throw if not booked ✗) · if (x) return true; return false; → just return x; · **reads need the lock too**, or a reader can see the seat added but the count not yet updated.

Q5 — lock granularity

synchronized on the method locks the whole registry: A1 and B7 bookings queue behind each other though they never conflict. Fine for small scale; a bottleneck on ticket-release day.

```
// Direction 1 – concurrent set + atomic add (no separate check)
private final Set<String> bookedSeats = ConcurrentHashMap.newKeySet();
public void book(String seatId) {
    if (!bookedSeats.add(seatId)) throw new SeatAlreadyBookedException(seatId);
}
public int getTotalBookings() { return bookedSeats.size(); } // derived → don't store it

// Direction 2 – per-seat locks, for multi-step booking (hold → pay → confirm)
Object lock = seatLocks.computeIfAbsent(seatId, k -> new Object());
synchronized (lock) { ... }
```

totalBookings is derived data (always the set's size) — the exercise 7 lesson. Storing it created a second piece of state to keep in sync; remove it and the inconsistency can't happen. Across multiple servers none of these in-memory locks work — you'd use a DB unique constraint or optimistic locking (Phase 4).

Q6 and Q7

Q6: return this.INSTANCE; inside a static method — **a static method has no this**. Write return INSTANCE; . Choose eager when construction is cheap and the instance is almost certainly used.

Q7 — the big one: you removed every synchronized when converting to @Service. Spring's default singleton scope means ONE SeatRegistry shared by every class — and every HTTP request runs on its own thread. A hundred users = a hundred threads calling book() on one object: the exact Part A bug. **Spring manages how many instances exist; it does nothing about thread safety inside them.** A very common real-world mistake — saying this unprompted is a strong signal.

Delete when converting to @Service	Keep
Holder class · getInstance() · the eager static final INSTANCE · private on the constructor (so tests can build it)	the fields · all methods · every synchronized · the exceptions

5. Mixed Exercise 1 — Ride Fare Engine

Clarifications you asked for (asking was itself the right move)

"Vehicle fare and ride fare are different? Which fare does the promo apply to?" There is **one fare per ride**, built in steps, and the promo is applied last:

1. vehicle fare = vehicle type's formula on the TripMeter totals (km, minutes)

2. surged fare = vehicle fare × surgeMultiplier(zone)

3. final fare = surged fare – promo discount (never below 0)

MINI, 10 km, 20 min, surge 1.5, FLAT50:

1. 40 + 15×10 + 2×20 = 230 2. 230 × 1.5 = 345 3. 345 – 50 = 295

Each step has a different owner (vehicle pricing, shared demand, marketing). **The order is a business rule and lives in one place** — the orchestrating service.

"Computed from the TripMeter totals... SEDAN minimum 150 — didn't understand." GPS pings add small amounts of distance and time to the ride's meter; when the ride ends you read the **totals** and plug them into the formula. SEDAN's minimum means "never less than 150": `return Math.max(150, 60 + 20*km + 3*minutes);` — a 1 km ride (formula 86) is charged 150. It's part of step 1, inside SEDAN's own pricing, before surge.

"What is a zone?" An area of the city (AIRPORT, OFFICE_PARK, MARKET...). Demand differs by place at the same moment, so the tracker keeps **one count per zone** — naturally a map, zone → count, shared by all threads. For the exercise, a zone is just a String.

Part A — the decisions (reference)

Req	Decision	Why
R1	Strategy	three variants of one algorithm, each with its own formula
R2	Factory (registry map)	type arrives as a runtime String; business logic must not name concrete classes. Strategies are stateless → share one instance each
R3	Singleton (or one injected instance) + thread-safe internals	"every part must see the SAME counts" = exactly one tracker; hundreds of threads = ConcurrentHashMap + AtomicInteger
R4	No pattern — plain class, fresh per ride	stateful per-ride data; sharing would mix rides
R5	No pattern for the format (plain formatter); interface in front of the SMS provider	one format, confirmed stable → YAGNI; but an external system gets an interface for testability
R6	Strategy + Factory	one promo today, but many are <i>confirmed</i> → a known change, not speculation
R7	Nothing to build — a constraint to keep in mind	in-memory state is only valid on one server (see Part C Q8)

R5 vs R6 — the key contrast: both are "one variant today". R5's future is confirmed **empty** → no pattern. R6's future is confirmed **full** → Strategy now. The deciding factor is whether the change is known, not how many variants exist today.

What you got right

✓ **Excellent demand-tracker concurrency** — learned yesterday, applied today:

```
private final ConcurrentHashMap<String, AtomicInteger> zoneDemands = new ConcurrentHashMap<>();
zoneDemands.computeIfAbsent(zoneId, k -> new AtomicInteger(0)).incrementAndGet();
getZoneCounter(zoneId).updateAndGet(k -> Math.max(0, k - 1)); // "not below 0" + decrement, atomically
```

✓ Ride created fresh per ride · ✓ DoubleAdder for thread-safe accumulation · ✓ SMS not over-abstracted.

What went wrong

- 1. No Singleton → R3 failed.** BookingApi did new SurgeMultiplier() in its constructor, and one BookingApi was made per ride — so every ride had a private tracker starting at zero, and surge was always 1.0. **The concurrency inside was perfect, protecting a counter nobody else could see.** The gap this exercise revealed: when patterns aren't labelled, you reached for Strategy and Factory but didn't map "everyone must see the same counts" to "exactly one instance".

2. The strategy did three jobs. calculateFare(km, minutes, surgeMultiplier, promo) forced every vehicle class to apply surge and promo — copied three times, and a marketing change edits vehicle classes. Strategy signature = only step 1: double baseFare(double km, double minutes).

3. R6 missing — 50 hardcoded as a double, despite confirmed future promo types.

4. Everything built with new in BookingApi — no constructor injection, nothing substitutable in tests. You've diagnosed this in three earlier exercises; it slipped back under time pressure.

5. Service mixed with per-ride state. BookingApi held vehicleType and ride as fields, so one object could handle one ride at a time — two concurrent rides overwrite each other. Split: a **stateless shared service** + **per-ride state passed in as parameters**.

Compile errors: `class AutoFareCalculator()` — no parentheses on class declarations, and missing `implements FareStrategy` · `'MINI'` single quotes (2nd time) and `switch (x):` → `switch (x) { ... }` · `count < 50` on an `AtomicInteger` → `count.get() < 50` · assigning `0` to final `DoubleAdder` fields in a constructor (delete it — a `DoubleAdder` starts at `0`) · `final Ride ride` assigned after construction · `private sendMessage(...)` no return type (6th time) · `vehicleFareCalculator.create, this.surgeMultiplier, dsurge, DoubleAdded, this.SmsMessage.sendMessage()` (class name used as a field, and the message not passed).

Reference orchestrator (full reference code is in the chat review)

```
interface FareStrategy { double baseFare(double km, double minutes); }
class SedanFare implements FareStrategy {
    private static final double MINIMUM_FARE = 150;
    public double baseFare(double km, double minutes) {
        return Math.max(MINIMUM_FARE, 60 + 20 * km + 3 * minutes);
    }
}
class FareStrategyFactory {
    private final Map<String, FareStrategy> strategies =
        Map.of("AUTO", new AutoFare(), "MINI", new MiniFare(), "SEDAN", new SedanFare());
    public FareStrategy forVehicle(String type) {
        FareStrategy s = strategies.get(type);
        if (s == null) throw new UnsupportedOperationException(type);
        return s;
    }
}
class FareService {
    // stateless, shared by all rides
    private final FareStrategyFactory fares;
    private final DemandTracker demand; // the ONE tracker
    private final PromoFactory promos;
    public FareService(FareStrategyFactory fares, DemandTracker demand, PromoFactory promos) { ... }

    public double calculate(String vehicleType, String zone, TripMeter meter, String promoCode) {
        double vehicleFare = fares.forVehicle(vehicleType).baseFare(meter.getKm(), meter.getMinutes());
        double surged = vehicleFare * demand.surgeMultiplier(zone);
        return promos.forCode(promoCode).apply(surged); // three lines = three steps, in order
    }
}
// surgeMultiplier reads with demand.get(zone), not computeIfAbsent — querying shouldn't create entries
```

6. Revision Tools

Your Phase 3 mistake summary

#	Habit	Count	Fix
1	Missing return types (<code>public getInstance()</code> , <code>private sendMessage()</code>)	6×	Every method: modifiers → return type → name. Only constructors have none.
2	Diagnosing a problem correctly but not applying the fix in code	4×	Re-read your critique; tick each point against a line of code.
3	<code>[]</code> on a List (<code>available[0]</code>)	6×	<code>.get(i)</code> / <code>.remove(i)</code>
4	Signatures drifting between interface and implementations	3×	Write the interface once; copy-paste signatures.
5	Incomplete requirements (cancel, Part B Checkout, Parts A/C skipped)	4×	List every requirement and tick it off.
6	A strategy/factory doing more than its job (inserting, exporting, applying surge)	3×	Strategies decide, factories return — the context acts.
7	<code>'X'</code> for Strings; <code>Default: ; instance of ; synchronize ; Int</code>	many	<code>"X" · default: · instanceof · synchronized · Integer</code>
8	Holder written as a method; missing <code>static</code> / <code>private</code> on Holder or constructor	2×	No <code>new</code> inside <code>getInstance()</code> ; <code>private static class Holder</code> ; <code>private constructor</code> .
9	Assuming a framework/collection handles concurrency (removed <code>synchronized</code> under <code>@Service</code> ; <code>volatile</code> on a Set)	2×	Shared object + mutable state = your job to protect it.
10	Not recognising a Singleton requirement when it isn't labelled	1×	"everyone sees the same..." / "must agree" / "exactly one" → one shared instance.

Requirement → pattern cheat sheet

If the requirement says...	Reach for
"supports X, Y and Z ways of doing..." , rules vary, algorithm swappable	Strategy
type arrives as a String/config at runtime; "business logic must not name..."	Factory (switch if small & fixed, registry if growing)
objects must come as a matching set ("never premium PDF with plain Excel")	Abstract Factory
"every part must see the same..." , "screens showed different numbers" , "exactly one pool/registry"	Singleton / one injected instance + thread-safe internals
per-request / per-ride / per-order data accumulates	No pattern — plain class, fresh per use
one variant, confirmed stable (one tax rate, one format)	No pattern — constant or plain class
"maybe one day..." , no ticket, no date	Nothing — mention the seam verbally
external system (SMS, payment gateway, DB, maps)	Interface even with one implementation (testability)

Interview quick-recall — Phase 3 so far

Cover the right column and answer out loud in one or two sentences.

Question	Answer in one breath
What is a design pattern?	A named, proven solution shape for a recurring design problem — vocabulary plus tested structure, not copyable code.
Name the three roles in Strategy.	Strategy (interface), ConcreteStrategy (implementations), Context (holds one and delegates).
How do strategies that need different data share one interface?	Each strategy carries its own configuration via its constructor; the interface stays minimal.
When would you NOT use Strategy?	One variant and none planned, or variants differing only by a number — use one parameterised class.
Strategy vs State?	Same structure; in Strategy the client picks, in State the states themselves trigger transitions.
Lambda or class for a strategy?	Lambda for stateless one-liners; class when it holds config/state, overrides hooks or needs a name and tests.
Strategy example in the JDK?	Comparator — different ordering algorithms passed to the same sort.
What should a strategy's method do?	Decide and return the decision; the context applies it. It shouldn't mutate the context's state.
Is an empty default method an LSP violation?	Only if the contract promises an effect. An optional notification hook can be ignored; a core operation can't.
What problem does Factory solve?	Keeps concrete class names and construction details out of business logic, concentrating creation in one place.
"You've just moved the if/else."	Yes — into one known place; business logic now depends only on the interface. A registry removes it entirely.
Why store Supplier instead of instances?	A Supplier gives a fresh object per call, needed when objects hold per-use state; instances are shared.
How do you decide shared vs fresh?	Is any field written after construction? No → share it. Yes → fresh per use (or synchronise).
Simple Factory vs Factory Method?	Simple picks by a parameter; Factory Method lets a subclass decide what to create.
What does Abstract Factory guarantee?	Families of related objects that must match — mismatches become impossible by construction.
Factory vs Builder?	Factory chooses which type; Builder assembles one complex object with many optional parts.
What are the two parts every Singleton needs?	A private constructor and a static accessor.
Why is naive lazy init broken?	Check-then-act: two threads both see null and both create an instance.
Why does double-checked locking need volatile?	The reference can be assigned before the constructor finishes; another thread could use a half-built object. volatile forbids that reordering.
Why two null checks in DCL?	The outer avoids locking once created; the inner handles two threads that both passed the outer check.
Why is Bill Pugh lazy and thread-safe?	Holder loads only when getInstance() first references it; class initialisation runs once under the JVM's lock.
Static field initialiser vs static method?	The initialiser runs once at class init; a method body runs every call.
Is object creation (new) locked?	No — only class initialisation is. Bill Pugh puts new inside a static initialiser so the JVM's lock covers it.
Best Singleton according to Bloch, and why?	Enum — thread-safe, and immune to reflection and serialisation by language design.
How can a Singleton be broken?	Reflection (setAccessible), serialisation, cloning, multiple classloaders. Defences: constructor guard, readResolve, clone throws.
Why is Singleton criticised?	Hidden dependency, untestable, violates DIP, global mutable state.
What do you do instead in Spring?	Let the container create one bean and inject it — same single instance, no global access.
Is a Spring singleton bean thread-safe?	No — it's shared across all request threads, so mutable state inside still needs protection.
Is a thread-safe singleton a thread-safe object?	No — only its creation is safe; its methods need their own protection.
What does volatile guarantee?	Visibility and ordering — never atomicity.
volatile vs synchronized?	volatile: everyone sees the latest value. synchronized: only one thread at a time, plus visibility.
Why is counter++ unsafe even with volatile?	It's read-add-write; threads interleave between the read and the write and lose an update.
volatile on a collection field?	Protects reassignment of the reference, not mutation of the contents.
ConcurrentHashMap vs synchronizedMap?	Fine-grained internal locking vs one lock on the whole map.
Is containsKey-then-put safe on a ConcurrentHashMap?	No — each call is safe but the sequence isn't. Use putIfAbsent / computeIfAbsent / merge.
What's lock granularity?	How much one lock guards. Coarse = one lock for everything; fine = per key, so unrelated work runs in parallel.
Should reads be synchronized?	Yes, when they read state written under a lock — otherwise they can see a half-finished update.
Why not store totalBookings?	It's derived (the set's size); storing it adds state that can go out of sync.

Phase 3 review checklist

Before writing code

- ☐ For each requirement: which pattern, or explicitly "none" — with one reason
- ☐ Any "everyone must see the same..." requirement? → one shared instance
- ☐ Which future changes are *confirmed* (abstract now) vs *speculative* (mention only)?
- ☐ Which objects are per-request state, and which are shared services?

Strategy / Factory

- ☐ Each strategy's signature takes only what *that* job needs
- ☐ Strategies decide and return; they don't mutate the context
- ☐ Configuration goes in the strategy's constructor
- ☐ Variants that differ only by a number → one parameterised class
- ☐ Factories *return* objects (return type is the interface, not `void`)
- ☐ No concrete class names or construction details in business logic
- ☐ Shared vs fresh decided deliberately per type
- ☐ Interface written once, signatures copied into implementations

Singleton / concurrency

- ☐ Private constructor + static accessor
- ☐ Bill Pugh: `private static class Holder` with a `static final` field; no `new` in `getInstance()`
- ☐ Every read-modify-write and check-then-act on shared state is atomic
- ☐ All methods touching shared state use the same lock — including reads
- ☐ No reliance on `volatile` for atomicity; no `volatile` + synchronized stacking
- ☐ Use a method's boolean result (`add` , `remove` , `putIfAbsent`) instead of check-then-act
- ☐ Derived values computed, not stored
- ☐ Spring bean still thread-safe inside

Syntax sweep

- ☐ Every method has a return type · `"Strings"` not `'x'` · default: lowercase
- ☐ `implements` on every strategy class · no parentheses after a class name
- ☐ No `this` in a static method · `.get(i)` not `[i]` · `instanceof` one word
- ☐ Every collaborator injected via the constructor, and actually used

Status: Strategy ✓ · Factory ✓ · Singleton ✓ · Mixed exercise 1 ✓ (reviewed) · Mixed exercise 2 in progress. Next patterns: **Builder** and **Observer**, then the jumbled exercise with no hints.